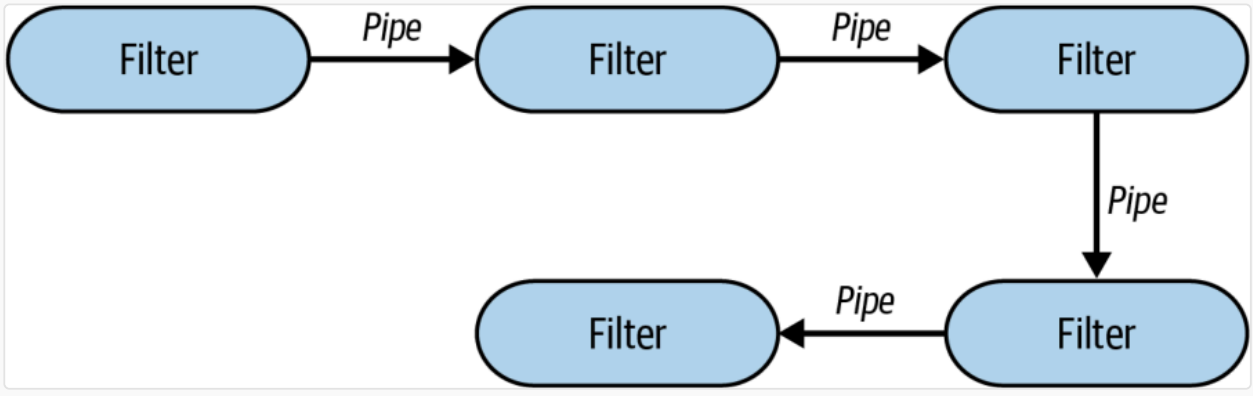
فصل ۱۲: سبک معماری خط لوله

یکی از سبک‌های بنیادین در معماری نرم‌افزار، معماری خط لوله است که معماری لوله‌ها و فیلترها (pipes and filters) نیز نامیده می‌شود. از همان زمانی که توسعه‌دهندگان و معماران تصمیم گرفتند کارکردها را به بخش‌های مجزا تقسیم کنند، این سبک معماری نیز پدید آمد. بیشترِ توسعه‌دهندگان این معماری را به‌عنوانِ اصلِ زیربناییِ زبان‌های پوستهٔ (shell) ترمینالِ Unix، مانندِ Bash و Zsh، می‌شناسند.

توسعه‌دهندگانی که با بسیاری از زبان‌های برنامه‌نویسی تابعی (functional programming) کار می‌کنند، شباهت‌هایی میانِ سازه‌های زبانی و عناصرِ این معماری خواهند دید. در واقع، بسیاری از ابزارهایی که از مدلِ برنامه‌نویسیِ MapReduce بهره می‌گیرند، از همین توپولوژیِ پایه پیروی می‌کنند. هرچند نمونه‌های این فصل پیاده‌سازی‌های سطح‌پایینِ سبک معماری خط لوله را نشان می‌دهند، این سبک را می‌توان برای برنامه‌های کسب‌وکاریِ سطح‌بالاتر نیز به کار برد.

توپولوژی

توپولوژیِ معماری خط لوله از دو نوعِ اصلیِ مؤلفه تشکیل می‌شود: لوله‌ها (pipes) و فیلترها (filters). فیلترها بخش‌های کارکردی سیستم را در بر می‌گیرند و وظایفِ کسب‌وکاریِ مشخصی را انجام می‌دهند، و لوله‌ها داده‌ها را به فیلترِ (یا فیلترهای) بعدی در زنجیره منتقل می‌کنند. این دو به شیوه‌ای مشخص با هم هماهنگ می‌شوند: لوله‌ها ارتباطی یک‌سویه و معمولاً نقطه‌به‌نقطه میانِ فیلترها پدید می‌آورند، چنان‌که در شکل ۱۲-۱ نشان داده شده است.



شکل ۱۲-۱. توپولوژیِ پایهٔ معماری خط لوله

بدین ترتیب، «شکلِ» هم‌ریختِ معماری خط لوله در قالبِ یک واحد استقرار تعریف می‌شود که کارکردهایش درونِ فیلترهایی جای دارند که با لوله‌های یک‌سویه (unidirectional) به هم متصل‌اند.

ویژگی‌های خاصِ سبک

هرچند بیشترِ پیاده‌سازی‌های معماری خط لوله تک‌پارچه‌اند، می‌توان هر فیلتر (یا مجموعه‌ای از فیلترها) را به‌صورتِ یک سرویس مستقر کرد و معماری‌ای توزیع‌شده با فراخوانی‌های دورِ هم‌گام یا ناهم‌گام به هر سرویس پدید آورد. توپولوژیِ استقرار هر چه باشد، این معماری فقط از دو مؤلفهٔ معماری تشکیل می‌شود، فیلترها و لوله‌ها، که در بخش‌های زیر به‌تفصیل شرح می‌دهیم.

فیلترها

فیلترها بخش‌هایی خودبسنده از کارکرد هستند که از فیلترهای دیگر استقلال دارند. فیلترها معمولاً بی‌حالت `(stateless)` هستند و باید تنها یک وظیفه را انجام دهند. کارهای مرکب معمولاً به‌جای یک فیلترِ واحد، با دنباله‌ای از فیلترها انجام می‌شوند.

از آنجا که فیلترها را می‌توان با بیش از یک فایلِ کلاس پیاده‌سازی کرد، مؤلفه‌های معماری به شمار می‌آیند (فصل ۸ را ببینید). حتی اگر فیلتری ساده باشد و فقط با یک فایلِ کلاسِ واحد پیاده‌سازی شده باشد، باز هم یک مؤلفه است.

در سبک معماری خط لوله چهار نوع فیلتر وجود دارد:

تولیدکننده `(Producer)`: نقطهٔ آغازِ یک فرایند است و فیلترهای تولیدکننده فقط خروجی دارند. گاهی به آن‌ها منبع `(source)` نیز می‌گویند. یک رابط کاربری و یک درخواستِ بیرونی به سیستم، هر دو نمونه‌هایی از فیلترهای تولیدکننده‌اند.

تبدیل‌گر `(Transformer)`: فیلترهای تبدیل‌گر ورودی را می‌پذیرند، در صورتِ نیاز روی بخشی از داده‌ها یا همهٔ آن‌ها تبدیلی انجام می‌دهند و سپس داده‌ها را به لولهٔ خروجی می‌فرستند. هواداران برنامه‌نویسی تابعی این ویژگی را با نامِ `map` می‌شناسند. فیلترهای تبدیل‌گر ممکن است، برای نمونه، داده‌ها را غنی‌سازی کنند، ساختار داده‌ها را تبدیل کنند یا نوعی محاسبه انجام دهند.

آزماینده `(Tester)`: فیلترهای آزماینده ورودی را می‌پذیرند، آن را بر پایهٔ یک یا چند معیار می‌آزمایند و سپس، در صورتِ نیاز، بر پایهٔ نتیجهٔ آزمون خروجی تولید می‌کنند. برنامه‌نویسانِ تابعی این را مشابهِ `reduce` می‌شناسند. یک فیلترِ آزماینده ممکن است بررسی کند که همهٔ داده‌ها معتبرند و درست وارد شده‌اند، یا مانندِ یک کلید عمل کند تا تعیین کند پردازش باید ادامه یابد یا نه (برای نمونه، «اگر مبلغِ سفارش کمتر از پنج دلار است، داده‌ها را به فیلترِ بعدی نفرست»).

مصرف‌کننده `(Consumer)`: نقطهٔ پایانِ جریانِ خط لوله است؛ فیلترهای مصرف‌کننده گاهی نتیجهٔ نهاییِ فرایندِ خط لوله را در یک پایگاه‌داده ماندگار می‌کنند یا نتایجِ نهایی را روی صفحهٔ رابط کاربری نمایش می‌دهند.

ماهیتِ یک‌سویه و سادگیِ لوله‌ها و فیلترها، بازاستفادهٔ ترکیبی `(compositional reuse)` را تشویق می‌کند. بسیاری از توسعه‌دهندگان این توانایی را با کار با پوسته‌ها کشف کرده‌اند. داستانِ مشهوری از وبلاگِ «`More Shell, Less Egg`» نشان می‌دهد که این انتزاع‌ها تا چه اندازه قدرتمندند. از `Donald Knuth` خواسته شد برنامه‌ای برای

حلِ این مسئلهٔ پردازشِ متن بنویسد: یک فایلِ متنی را بخوان، n واژهٔ پرکاربردتر را تعیین کن و فهرستی از آن واژه‌ها را به ترتیبِ بسامد چاپ کن. او برنامه‌ای بیش از ۱۰ صفحه به زبانِ Pascal نوشت و در این میان الگوریتمِ تازه‌ای طراحی (و مستند) کرد. سپس Doug McIlroy یک اسکریپتِ پوسته (shell script) را نشان داد که آن‌قدر کوچک بود که به‌راحتی در یک پستِ شبکه‌های اجتماعی جا می‌شد و مسئله را به شکلی زیبا حل می‌کرد:

```
tr -cs A-Za-z '\n' |
tr A-Z a-z |
sort |
uniq -c |
sort -rn |
sed ${1}q
```

حتی طراحانِ پوسته‌های Unix نیز اغلب از کاربردهای خلاقانه‌ای که توسعه‌دهندگان برای انتزاع‌های ترکیبیِ ساده اما قدرتمندِ آن‌ها می‌یابند، شگفت‌زده می‌شوند.

لوله‌ها

لوله‌ها در این معماری کانالِ ارتباطی میانِ فیلترها را تشکیل می‌دهند. هر لوله معمولاً یک‌سویه و نقطه‌به‌نقطه است: ورودی را از یک منبع می‌پذیرد و خروجی را به مقصدی دیگر هدایت می‌کند. محمولهٔ (payload) لوله‌ها می‌تواند هر قالبِ داده‌ای باشد، اما معماران معمولاً حجمِ کمترِ داده را ترجیح می‌دهند تا کاراییِ بالا ممکن شود.

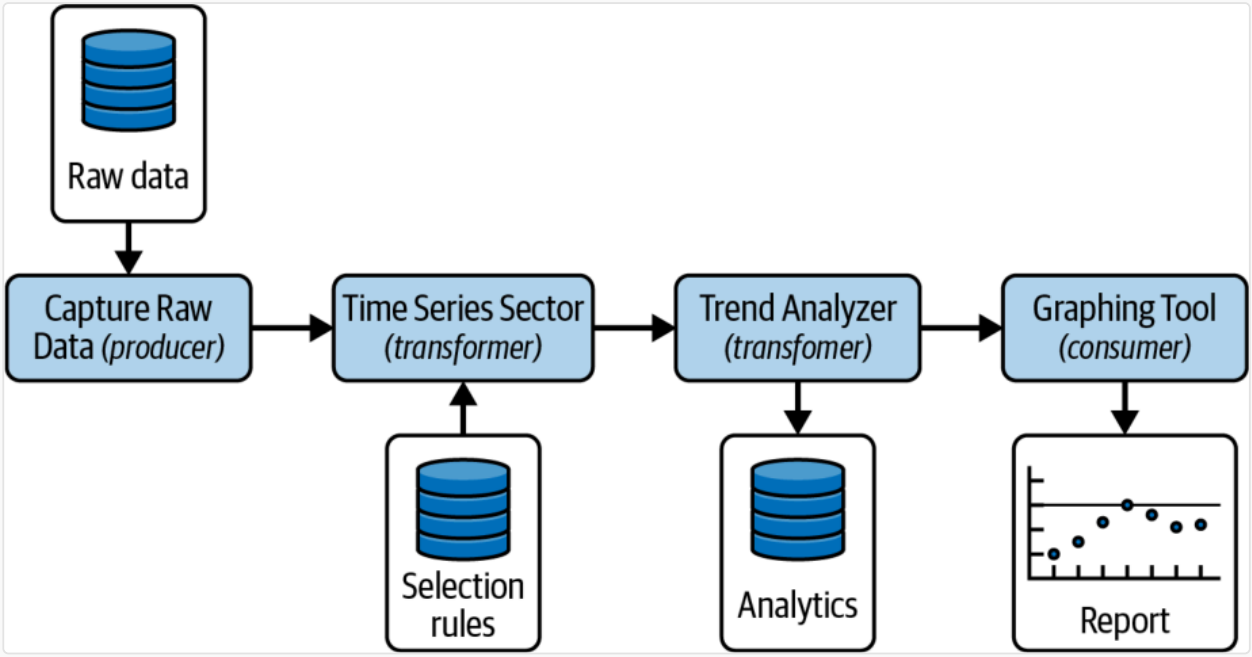
اگر یک فیلتر (یا گروهی از فیلترها) به‌صورتِ یک سرویسِ جداگانه و به شیوهٔ توزیع‌شده مستقر شود، لوله‌ها با استفاده از REST، پیام‌رسانی، جریان‌سازیِ داده (streaming) یا پروتکلِ ارتباطِ دورِ دیگری، یک فراخوانیِ دورِ یک‌سویه انجام می‌دهند. توپولوژیِ استقرارشان چه تک‌پارچه باشد چه توزیع‌شده، لوله‌ها می‌توانند هم‌گام یا ناهم‌گام باشند. در استقرارهای تک‌پارچه، معماران برای ارتباطِ ناهم‌گام با یک فیلتر از ریسمان‌ها یا پیام‌رسانیِ تعبیه‌شده (embedded messaging) استفاده می‌کنند.

توپولوژی‌های داده

از آنجا که بیشترِ معماری‌های خط لوله به‌صورتِ تک‌پارچه مستقر می‌شوند، معمولاً با یک پایگاه‌دادهٔ تک‌پارچهٔ واحد همراه هستند. اما همیشه چنین نیست. توپولوژیِ پایگاه‌داده در این سبک معماری ممکن است تفاوتِ چشمگیری داشته باشد: از یک پایگاه‌دادهٔ واحد تا یک پایگاه‌داده برای هر فیلتر.

نمونهٔ شکل ۱۲-۲ معماری خط لوله‌ای را برای یک تابع برازشِ مداوم (continuous fitness function) متداول (یعنی یک آزمونِ معماری) نشان می‌دهد که در محیطِ تولید اجرا می‌شود و یک ویژگی عملیاتیِ مشخص (مانندِ پاسخ‌گویی یا مقیاس‌پذیری) را تحلیل می‌کند. توجه کنید که فیلترِ Capture Raw Data یک پایگاه‌دادهٔ جداگانه را که داده‌های خام را در خود دارد، بارگذاری می‌کند. این فیلتر داده‌های خام را از راهِ یک لوله به فیلترِ Time Series Selector می‌فرستد که اطلاعاتِ پیکربندی (مانندِ بازهٔ زمانیِ تحت تحلیل) را از پایگاه‌داده‌ای جداگانه می‌خواند.

داده‌های تبدیل‌شده سپس از راهِ لولهٔ دیگری به فیلترِ `Trend Analyzer` فرستاده می‌شوند که داده‌ها را تحلیل می‌کند و نتایجِ تحلیل را در پایگاه‌داده‌ای جداگانه ذخیره می‌کند. این فیلتر سپس داده‌های تحلیلی را از راهِ آخرین لوله به فیلترِ `Graphing Tool` می‌فرستد که با تولیدِ گزارشی نموداری از داده‌ها، خط لوله را به پایان می‌رساند.



شکل ۱۲-۲. معماری‌های خط لوله می‌توانند یک پایگاه‌داده یا چندین پایگاه‌داده داشته باشند

ملاحظات ابری

سبک معماری خط لوله، به دلیلِ سطحِ بالای ماژولاریتی و انواعِ جداگانهٔ فیلترهایش، برای استقرارهای مبتنی بر ابر بسیار مناسب است. از آنجا که بیشترِ معماری‌های خط لوله چندان پیچیده و گسترده نیستند، به‌خوبی در قالبِ معماری‌های تک‌پارچه پیاده‌سازی می‌شوند؛ یعنی همهٔ فیلترها در یک واحد استقرار قرار می‌گیرند.

اما فیلترها در محیط‌های ابری به‌صورتِ توابعِ توزیع‌شده نیز به‌خوبی کار می‌کنند. در `AWS`، معماری خط لوله را می‌توان به‌صورتِ `AWS Step Functions` مستقر کرد که در آن هر فیلتر به‌صورتِ یک `lambda` جداگانه در گردش‌کار مستقر می‌شود. `AWS Step Functions` دو گردش‌کار عرضه می‌کند: `Standard`، که در آن هر گام دقیقاً یک بار اجرا می‌شود، و `Express`، که در آن هر گام ممکن است بیش از یک بار اجرا شود. سبک معماری خط لوله با هر دو کار می‌کند. کدِ زیر نمونهٔ تابع برازشِ مداومِ شکل ۱۲-۲ را به‌عنوانِ یک استقرارِ ابریِ متداول برای معماری خط لوله نشان می‌دهد که به‌صورتِ یک `AWS Step Function` پیاده‌سازی شده است:

```
{
  "Comment": "Measure and analyze scalability trends.",
  "StartAt": "Capture Raw Data",
  "States": {
    "Capture Raw Data": {
    "Type": "Task",
    "Resource": "arn:aws:lambda:region:account_id:function:raw_data_capture",
    "Next": "Time Series Selector"
   },
    "Time Series Selector": {
    "Type": "Task",
    "Resource": "arn:aws:lambda:region:account_id:function:time_series_selector",
    "Next": "Trend Analyzer"
   },
    "Trend Analyzer": {
    "Type": "Task",
    "Resource": "arn:aws:lambda:region:account_id:function:trend_analyzer",
    "Next": "Graphing Tool"
    },
    "Graphing Tool": {
    "Type": "Task",
    "Resource": "arn:aws:lambda:region:account_id:function:graphing_tool",
    "End": true
    }
  }
}
```

این نمونه تنها راهِ به‌کارگیریِ معماری خط لوله در محیط‌های ابری نیست. فیلترها را می‌توان به‌صورتِ توابعِ بدون سرور `(serverless functions)`، توابعِ کانتینری `(containerized functions)` یا حتی یک سرویسِ واحد که هر چهار مؤلفهٔ فیلتر را در یک استقرارِ تک‌پارچه در بر می‌گیرد نیز مستقر کرد.

ریسک‌های رایج

هدفِ اصلیِ معماری خط لوله جداسازیِ کارکردها در قالبِ فیلترهای تک‌منظوره `(single-purpose filters)` است؛ فیلترهایی که هر یک عملِ مشخصی روی داده‌ها انجام می‌دهند و سپس آن‌ها را برای پردازشِ بیشتر به فیلترِ دیگری تحویل می‌دهند. از این رو، یکی از رایج‌ترین ریسک‌های این معماری، بارکردنِ مسئولیتِ بیش از حد بر دوشِ فیلترهاست. حاکمیتِ خوب، که در بخشِ بعد به آن می‌پردازیم، با شناساییِ هدفِ هر مؤلفهٔ فیلتر به تیم‌ها کمک می‌کند این ریسک را کاهش دهند.

ریسکِ رایجِ دیگر در این سبک معماری، واردکردنِ ارتباطِ دوسویه `(bidirectional communication)` میانِ فیلترهاست. لوله‌ها قرار است فقط یک‌سویه باشند تا جداسازیِ روشنی از دغدغه‌ها میانِ فیلترها فراهم کنند و از همکاریِ میانِ آن‌ها جلوگیری شود. اگر معلوم شود ارتباطِ دوسویه ضروری است، این نشانهٔ روشنی است که معماری خط لوله احتمالاً سبکِ درستی برای استفاده نیست، یا فیلترها بیش از حد پیچیده‌اند و کارکردها به‌درستی مرزبندی نشده‌اند.

رسیدگی به وضعیت‌های خطا (error conditions) پیچیدگیِ دیگری است که می‌تواند ریسکِ چشمگیری در این سبک معماری پدید آورد. اگر خطایی درونِ یک خط لوله رخ دهد، تعیینِ اینکه چگونه باید به‌درستی از خط لوله خارج شد و پس از شروع پردازش سیستم را بازیابی کرد، اغلب دشوار است. به همین دلیل، مهم است که معماران پیش از تعریفِ معماری، همهٔ وضعیت‌های خطای مهلکِ (fatal error) ممکن درونِ خط لوله را مشخص کنند.

آخرین حوزهٔ ریسک، مدیریتِ قراردادهای میانِ فیلترهاست. هر لوله قراردادی دارد که نمایانگرِ داده‌هایی (و احتمالاً نوع‌های متناظرشان) است که به فیلترِ بعدی می‌فرستد. تغییرِ یک قرارداد میانِ فیلترها به حاکمیت و آزمونِ سخت‌گیرانه نیاز دارد تا اطمینان حاصل شود فیلترهای دیگری که این قرارداد را دریافت می‌کنند، از کار نمی‌افتند.

حاکمیت

حاکمیت بر ویژگی‌های عملیاتیِ عمومی، مانندِ پاسخ‌گویی، مقیاس‌پذیری، در دسترس‌بودن و مانندِ آن‌ها، به هر موردِ کاربردِ مشخص وابسته است و ممکن است تفاوتِ بسیاری داشته باشد. اما از منظرِ ساختاری، معماران برای حاکمیت بر معماری‌های خط لوله از نقش و مسئولیتِ هر نوعِ فیلتر بهره می‌گیرند.

هر یک از چهار نوعِ پایه‌ایِ فیلتر (تولیدکننده، تبدیل‌گر، آزماینده و مصرف‌کننده) نقشِ مشخصی ایفا می‌کند. اما برای توسعه‌دهندگان بسیار آسان است که مسئولیتِ بیش از حدی بر دوشِ فیلترها بگذارند و معماری خط لوله را به یک تک‌پارچهٔ بی‌ساختار تبدیل کنند.

نوشتنِ یک تابع برازشِ خودکار برای حاکمیت بر این موضوع دشوار است که آیا یک فیلترِ تولیدکننده واقعاً نقطهٔ آغازِ خط لوله است، یا آیا یک فیلترِ آزماینده واقعاً یک وارسیِ شرطی برای تعیینِ ادامه یا پایانِ جریان انجام می‌دهد. فنونِ حاکمیت به معماران کمک می‌کنند تیمِ توسعه را در پایبندی به نقشِ هر نوعِ فیلتر و مسئولیتِ آن راهنمایی کنند.

یکی از این فنون، بهره‌گیری از برچسب‌ها (tags) است. (در زبانِ Java برچسب‌ها به‌صورتِ حاشیه‌نویسی‌ها (annotations) و در C# به‌صورتِ صفت‌های سفارشی (custom attributes) پیاده‌سازی می‌شوند.) برچسب‌ها در واقع هیچ کارکردی انجام نمی‌دهند؛ آن‌ها به شیوه‌ای برنامه‌نویسانه فراداده‌ای (metadata) دربارهٔ مؤلفه یا سرویس فراهم می‌کنند. به‌کارگیریِ برچسب‌ها در کلاسِ آغازینِ مؤلفهٔ فیلتر به توسعه‌دهندگان می‌گوید چه نوع فیلتری را می‌سازند یا تغییر می‌دهند و کمک می‌کند از سپردنِ مسئولیتِ بیش از حد به آن فیلتر یا انجامِ کارکردی که به نوعِ آن مربوط نیست، پرهیز کنند.

برای نشان دادنِ این فن، کدِ منبعِ زیر را در نظر بگیرید که برچسب‌هایی را برای چهار نوعِ پایه‌ایِ فیلتر تعریف می‌کند. این تعریفِ حاشیه‌نویسیِ برچسب در Java است:

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface Filter {
   public FilterType[] value();
   public enum FilterType {
      PRODUCER,
      TESTER,
      TRANSFORMER,
      CONSUMER
   }
}
```

و این تعریفِ صفتِ سفارشیِ برچسب در `C#` است:

```
[System.AttributeUsage(System.AttributeTargets.Class)]
class Filter : System.Attribute {
   public FilterType[] filterType;
   public enum FilterType {
      PRODUCER,
      TESTER,
      TRANSFORMER,
      CONSUMER
   };
}
```

از آنجا که فیلترها در اصل مؤلفه‌هایی معماری هستند، می‌توان آن‌ها را با چندین فایلِ کلاس پیاده‌سازی کرد. برای نمونه، در مثالِ تابع برازشِ مداوم در شکل ۱۲-۲، فیلترِ `Trend Analyzer` نسبتاً پیچیده است و ممکن است چندین فایلِ کلاس داشته باشد. به همین دلیل، یک برچسبِ دیگر تعریف می‌کنیم تا کلاسی را که نقطهٔ ورودِ `(entry point)` فیلتر را بازنمایی می‌کند شناسایی کنیم و بتوانیم برچسب‌های دیگر را به آن پیوست کنیم.

این تعریفِ برچسبِ نقطهٔ ورود در `Java` است:

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface FilterEntrypoint {}
```

و این تعریفِ برچسبِ نقطهٔ ورود در `C#` :

```
[System.AttributeUsage(System.AttributeTargets.Class)]
class FilterEntrypoint : System.Attribute {}
```

اکنون توسعه‌دهندگان می‌توانند برچسبِ `FilterType` را به کلاسِ `Entrypoint` بیفزایند و نوعِ آن و نقشِ متناظرش در معماری را مشخص کنند. برای نمونه، حاشیه‌نویسی‌های زیر نوع و نقشِ فیلترِ تبدیل‌گرِ `Trend Analyzer` را مشخص می‌کنند. در `Java` :

```
@FilterEntrypoint
@Filter(FilterType.TRANSFORMER)
public class TrendAnalyzerFilter {
   ...
}
```

و در `C#`:

```
[FilterEntrypoint]
[Filter(FilterType.TRANSFORMER)]
class TrendAnalyzerFilter {
   ...
}
```

هرچند این فن ممکن است نتواند جلوی همهٔ توسعه‌دهندگان را بگیرد که یک فیلترِ از نوعِ تبدیل‌گر را به اجرای منطقِ آزمون (کاری که باید فیلترِ آزماینده انجام دهد) وادار کنند، دست‌کم زمینه و شفافیتِ مفهومیِ بیشتری فراهم می‌کند.

ملاحظاتِ توپولوژی تیم

برخلافِ برخی از سبک‌های معماریِ شرح‌داده‌شده در این کتاب، سبک معماری خط لوله به‌طورِ کلی مستقل از توپولوژی‌های تیم است و با هر پیکربندیِ تیمی کار می‌کند:

تیم‌های هم‌راستا با جریان: از آنجا که معماری خط لوله معمولاً کوچک و خودبسنده است و یک سفر یا جریانِ واحد را در سیستم بازنمایی می‌کند، با تیم‌های هم‌راستا با جریان به‌خوبی کار می‌کند. در این توپولوژیِ تیم، تیم‌ها معمولاً مالکِ جریان در سیستم از آغاز تا پایان‌اند و این با شکلِ معماری خط لوله به‌خوبی جور درمی‌آید.

تیم‌های توانمندساز: از آنجا که معماری خط لوله بسیار ماژولار است و بر پایهٔ دغدغه‌های فنی جدا شده است، به‌خوبی با توپولوژی‌های تیمِ توانمندساز جفت می‌شود. متخصصان و اعضای میان‌بُرشی تیم می‌توانند با افزودنِ فیلترهای تازه درونِ خط لوله، بی‌آنکه بر بقیهٔ جریان اثر بگذارند، پیشنهادهایی بدهند و آزمایش‌هایی انجام دهند. برای نمونه، در مثالِ تابع برازشِ مداوم، یک تیمِ توانمندساز ممکن است پس از فیلترِ `Time Series Selector` یک فیلترِ تبدیل‌گر بیفزاید تا یک تحلیلِ روندِ جایگزین انجام دهد. این فیلترِ تازه از همان داده‌های فیلترِ موجودِ `Trend Analyzer` استفاده می‌کند، بی‌آنکه جریانِ عادیِ خط لوله را مختل کند.

تیم‌های زیرسیستم پیچیده: از آنجا که هر فیلتر کارِ بسیار مشخصی انجام می‌دهد، معماری خط لوله با توپولوژیِ تیمِ زیرسیستم پیچیده به‌خوبی کار می‌کند. اعضای مختلفِ تیم می‌توانند مستقل از یکدیگر (و از فیلترهای دیگر) بر پردازشِ پیچیدهٔ فیلترها تمرکز کنند. تحویل‌های یک‌سویهٔ داده در این سبک معماری به اعضای تیم‌های زیرسیستم پیچیده امکان می‌دهد با تمرکزی دقیق به پیچیدگیِ درونِ پردازشِ فیلترِ مشخصِ خود بپردازند.

تیم‌های پلتفرم: تیم‌های پلتفرمی که روی یک معماری خط لوله کار می‌کنند، می‌توانند با به‌کارگیریِ ابزارها، سرویس‌ها، `API` ها و کارهای مشترک، از درجهٔ بالای ماژولاریتیِ آن بهره ببرند.

ویژگی‌های سبک

در جدول رتبه‌بندی ویژگی‌ها (شکل ۱۲-۳)، رتبهٔ یک‌ستاره یعنی آن ویژگی معماریِ مشخص در این معماری به‌خوبی پشتیبانی نمی‌شود، در حالی که رتبهٔ پنج‌ستاره یعنی آن ویژگی معماری یکی از قوی‌ترین جنبه‌های این سبک معماری است. ویژگی‌هایی که در این کارنامه آمده‌اند در فصل ۴ شرح و تعریف شده‌اند.

سبک معماری خط لوله معماری‌ای افرازشدهٔ فنی است، زیرا منطقِ برنامه در آن بر پایهٔ انواعِ فیلتر جدا می‌شود. همچنین، از آنجا که معماری خط لوله معمولاً به‌صورتِ استقرارِ تک‌پارچه پیاده‌سازی می‌شود، کوانتوم معماری‌اش همیشه ۱ است.

	Architectural characteristic	Star rating
	Overall cost	$
Structural	Partitioning type	Technical
	Number of quanta	1
	Simplicity	★★★★
	Modularity	★★
Engineering	Maintainability	★★
	Testability	★★★
	Deployability	★★
	Evolvability	★★★
Operational	Responsiveness	★★★
	Scalability	★
	Elasticity	★
	Fault tolerance	★

شکل ۱۲-۳. رتبه‌بندیِ ویژگی‌های معماری خط لوله

هزینهٔ کلی، سادگی و ماژولاریتی نقاطِ قوتِ اصلیِ سبک معماری خط لوله‌اند. معماری‌های خط لوله، از آنجا که تک‌پارچه‌اند، پیچیدگی‌های سبک‌های معماری توزیع‌شده را ندارند: ساده‌اند، فهمشان آسان است و ساخت و نگه‌داری‌شان نسبتاً کم‌هزینه است. ماژولاریتیِ معماری از راهِ جداسازی دغدغه‌ها میانِ انواعِ گوناگونِ فیلتر و تبدیل‌گرها به دست می‌آید: هر فیلتری را می‌توان بدونِ اثرگذاری بر فیلترهای دیگر تغییر داد یا جایگزین کرد. برای نمونه، در مثالِ `Kafka` که در شکل ۱۲-۴ نشان داده شده است، می‌توان `Duration Calculator` را برای تغییرِ شیوهٔ محاسبهٔ مدت‌زمان اصلاح کرد، بی‌آنکه فیلترِ دیگری تغییر کند.

استقرارپذیری و آزمون‌پذیری، هرچند فقط در حدِ متوسط‌اند، به دلیلِ سطحِ ماژولاریتی‌ای که فیلترها می‌توانند به آن برسند، اندکی بالاتر از معماری لایه‌ای رتبه می‌گیرند. اما معماری‌های خط لوله همچنان معمولاً تک‌پارچه‌اند؛ بنابراین کاستی‌هایشان شاملِ تشریفات، ریسک، بسامدِ استقرار و کامل‌بودنِ آزمون است.

کشسانی و مقیاس‌پذیری در معماری خط لوله رتبهٔ بسیار پایینی (یک‌ستاره) دارند که دلیلِ اصلی‌اش استقرارهای تک‌پارچه است. پیاده‌سازیِ این سبک معماری به‌صورتِ یک معماری توزیع‌شده با ارتباطِ ناهم‌گام می‌تواند این ویژگی‌ها را به‌طورِ چشمگیری بهبود دهد، اما بهای آن ضربه‌ای به هزینهٔ کلی و سادگی است.

معماری‌های خط لوله از تحمل خطا پشتیبانی نمی‌کنند، زیرا معمولاً به‌صورتِ سیستم‌های تک‌پارچه مستقر می‌شوند. اگر بخشِ کوچکی از یک معماری خط لوله دچارِ خطای کمبود حافظه شود، کلِ واحدِ برنامه آسیب می‌بیند و از کار می‌افتد. افزون بر این، در دسترس‌بودنِ کلی از میانگین زمانِ بازیابیِ بالا، که در بیشترِ برنامه‌های تک‌پارچه رایج است، آسیب می‌بیند؛ زمانِ راه‌اندازی بر حسب دقیقه سنجیده می‌شود. همانند کشسانی و مقیاس‌پذیری، پیاده‌سازیِ این سبک معماری به‌صورتِ یک معماری توزیع‌شده با ارتباطِ ناهم‌گام می‌تواند تحمل خطا را به‌طورِ چشمگیری بهبود دهد؛ باز هم با پرداختِ بهای آن در هزینه و پیچیدگی.

چنان‌که گفتیم، بیشترِ ویژگی‌های عملیاتیِ کم‌امتیاز را می‌توان با تبدیلِ این معماری به یک معماری توزیع‌شده با ارتباطِ ناهم‌گام ارتقا داد؛ معماری‌ای که در آن هر فیلتر یک واحد استقرارِ جداگانه است و لوله‌ها فراخوانی‌های دورند. اما این کار بر ویژگی‌های دیگری، مانندِ سادگی و هزینه، اثرِ منفی می‌گذارد و یکی از مصالحه‌های کلاسیکِ معماری نرم‌افزار را به نمایش می‌گذارد.

چه زمانی استفاده کنیم

معماری خط لوله برای سیستم‌هایی با هر میزان پیچیدگی که گام‌های پردازشیِ متمایز، مرتب، قطعی `(deterministic)` و یک‌سویه دارند، مناسب است. سادگیِ این سبک آن را برای موقعیت‌هایی با محدودیت‌های شدید زمانی و بودجه‌ای نیز بسیار مناسب می‌کند.

چه زمانی استفاده نکنیم

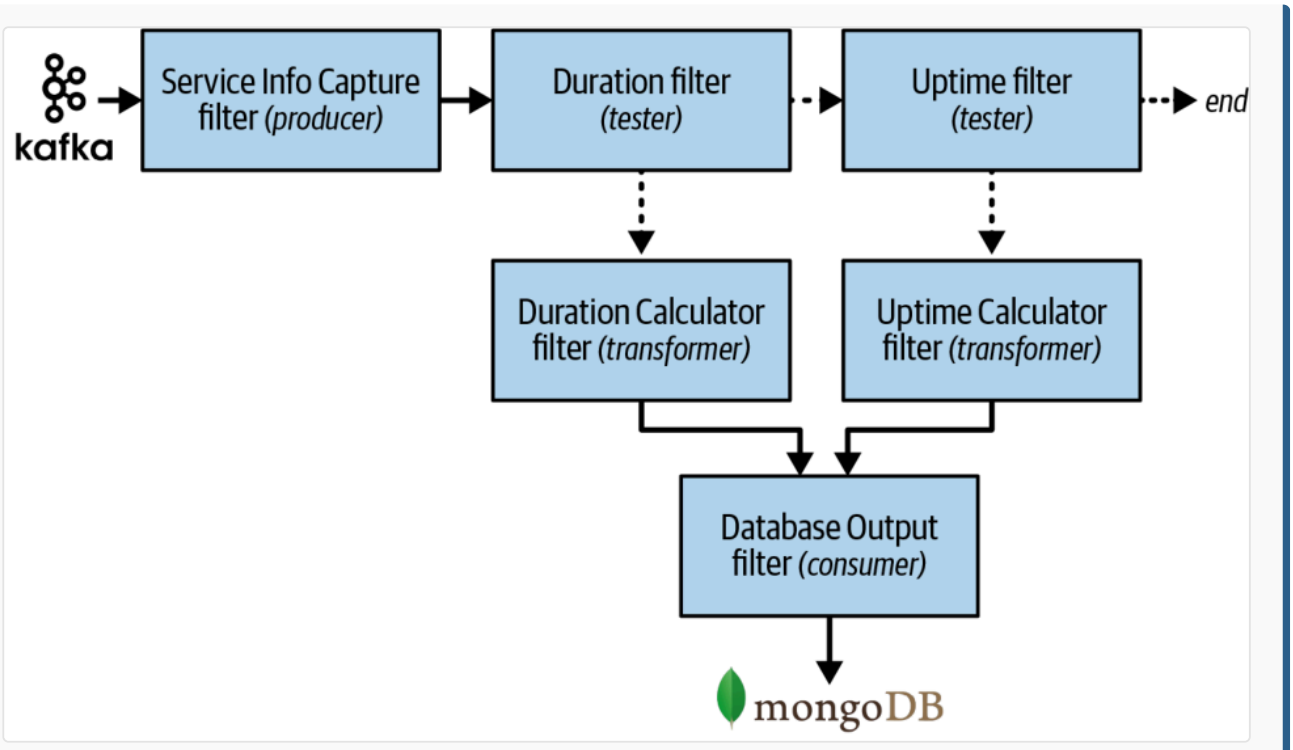
این سبک معماری، به دلیلِ ماهیتِ تک‌پارچه‌اش، برای سیستم‌هایی که به مقیاس‌پذیری، کشسانی و تحمل خطای بالا نیاز دارند مناسب نیست. اما به‌کارگیریِ رویکردِ معماری توزیع‌شده به کاهشِ این نگرانی‌ها کمک می‌کند.

سبک معماری خط لوله همچنین برای سناریوهایی که شاملِ ارتباطِ رفت‌وبرگشتی `(back-and-forth communication)` میانِ فیلترها هستند مناسب نیست، زیرا لوله‌ها قرار است فقط یک‌سویه باشند. هرچند می‌توان با به‌کارگیریِ شمارِ زیادی فیلترِ آزماینده در سراسرِ خط لوله، از این سبک معماری برای گردش‌کارهای غیرقطعی `(nondeterministic workflows)` استفاده کرد، چنین کاری را توصیه نمی‌کنیم. این کار این سبک معماریِ نسبتاً ساده را بیش از حد پیچیده می‌کند و ممکن است بر قابلیت نگهداری، آزمون‌پذیری، استقرارپذیری و در نتیجه بر قابلیت اطمینانِ کلی اثرِ منفی بگذارد. معماری رویدادمحور (فصل ۱۵ را ببینید) برای موقعیت‌هایی که شاملِ گردش‌کارهای غیرقطعی‌اند بسیار مناسب‌تر است.

سبک معماری خط لوله در برنامه‌های گوناگونی پدیدار می‌شود، به‌ویژه در کارهایی که پردازشِ ساده و یک‌سویه را تسهیل می‌کنند. برای نمونه، بسیاری از ابزارهای تبادلِ الکترونیکیِ داده (electronic data interchange یا `EDI`) از این الگو استفاده می‌کنند و با لوله‌ها و فیلترها تبدیل‌هایی از یک نوعِ سند به نوعی دیگر می‌سازند. ابزارهای استخراج، تبدیل و بارگذاریِ (extract, transform, and load یا `ETL`) پایگاه‌داده از معماری‌های خط لوله بهره می‌گیرند تا داده‌ها را تغییر دهند و از یک پایگاه‌داده یا منبعِ داده به دیگری روان کنند. هماهنگ‌کننده‌ها و میانجی‌هایی مانندِ `Apache Camel` از معماری خط لوله برای انتقالِ اطلاعات از یک گام به گامِ دیگر در یک فرایندِ کسب‌وکاری استفاده می‌کنند.

نمونه‌ها و موارد کاربرد

برای نشان دادنِ چگونگیِ به‌کارگیریِ معماری خط لوله، نمونهٔ زیر را در نظر بگیرید که در آن اطلاعاتِ تله‌متریِ `(telemetry)` سرویس‌ها از راهِ جریان‌سازی داده از سرویس‌ها به `Apache Kafka` فرستاده می‌شود (شکل ۱۲-۴).



شکل ۱۲-۴. نمونه‌ای از معماری خط لوله

سیستمِ نشان‌داده‌شده در شکل ۱۲-۴ از سبک معماری خط لوله برای پردازشِ انواعِ گوناگونِ داده‌هایی که به `Kafka` جریان می‌یابند استفاده می‌کند. فیلترِ `Service Info Capture` (فیلترِ تولیدکننده) مشترکِ موضوعِ `Kafka` می‌شود و اطلاعاتِ سرویس را دریافت می‌کند. سپس این داده‌های گردآوری‌شده را به یک فیلترِ آزماینده به نامِ فیلترِ `Duration` می‌فرستد تا تعیین کند آیا این داده‌ها به مدت‌زمانِ (بر حسبِ میلی‌ثانیه) درخواستِ سرویس مربوط‌اند یا نه.

به جداسازی دغدغه‌ها میانِ فیلترها توجه کنید: فیلترِ `Service Info Capture` فقط به این می‌پردازد که چگونه به یک موضوعِ `Kafka` متصل شود و داده‌های جریانی را دریافت کند، در حالی که فیلترِ `Duration` فقط به ارزیابیِ داده‌ها و تعیینِ اینکه آیا آن‌ها را به لولهٔ بعدی بفرستد یا نه، می‌پردازد. اگر داده‌ها به مدت‌زمانِ درخواستِ سرویس مربوط باشند، به فیلترِ تبدیل‌گرِ `Duration Calculator` فرستاده می‌شوند؛ در غیرِ این صورت، داده‌ها به فیلترِ آزمایندهٔ `Uptime` فرستاده می‌شوند تا بررسی شود آیا به معیارهای زمانِ فعالیت `(uptime)` مربوط‌اند یا نه. اگر نباشند، خط لوله پایان می‌یابد؛ این داده‌ها برای این جریانِ پردازشیِ خاص اهمیتی ندارند. اگر به معیارهای زمانِ فعالیت مربوط باشند، فیلترِ آزمایندهٔ `Uptime` آن‌ها را به `Uptime Calculator` می‌فرستد که از آن‌ها برای محاسبهٔ معیارهای زمانِ فعالیت استفاده می‌کند. `Uptime Calculator` سپس داده‌های تغییریافته را به مصرف‌کنندهٔ `Database Output` می‌فرستد که آن‌ها را در یک پایگاه‌دادهٔ `MongoDB` ماندگار می‌کند.

این نمونه نشان می‌دهد معماری خط لوله تا چه اندازه گسترش‌پذیر است. برای نمونه، در شکل ۱۲-۴، به‌آسانی می‌توان پس از فیلترِ `Uptime` یک فیلترِ آزمایندهٔ تازه افزود تا داده‌ها را برای استفاده در یک معیارِ جدید، مانندِ زمانِ انتظارِ اتصال به پایگاه‌داده، بفرستد.

سبک معماری خط لوله، هرچند معمولاً معماری‌ای تک‌پارچه است، به لطفِ به‌کارگیریِ فیلترهای افرازشدهٔ فنی سطحِ خوبی از ماژولاریتی را نشان می‌دهد. این سبک برای موقعیت‌هایی که شاملِ رویکردی گام‌به‌گام و مبتنی بر گردش‌کار برای پردازشِ داده‌ها درونِ یک سیستم‌اند بسیار مناسب است.

پیش از آنکه به سراغِ معماری‌های توزیع‌شدهٔ پیچیده‌تر برویم، یک معماری تک‌پارچهٔ دیگر باقی مانده است که در فصلِ بعد شرح داده می‌شود؛ معماری‌ای که با به‌کارگیریِ مؤلفه‌های افزونه سطحِ خوبی از ماژولاریتی را پشتیبانی می‌کند: معماری میکروکرنل.

واژه‌نامهٔ اختصاصی این فصل

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Pipeline Architecture	معماری خط لوله
Pipes and Filters Architecture	معماری لوله‌ها و فیلترها
Pipe	لوله
Filter	فیلتر
Shell	پوسته
Shell Script	اسکریپت پوسته
Functional Programming	برنامه‌نویسی تابعی

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Unidirectional	یک‌سویه
Stateless	بی‌حالت
Producer (filter)	تولیدکننده
Source (filter)	منبع
Transformer (filter)	تبدیل‌گر
Tester (filter)	آزماینده
Consumer (filter)	مصرف‌کننده
Compositional Reuse	بازاستفادهٔ ترکیبی
Payload	محموله
Streaming	جریان‌سازی داده
Embedded Messaging	پیام‌رسانی تعبیه‌شده
Continuous Fitness Function	تابع برازش مداوم
Serverless Function	تابع بدون سرور
Containerized Function	تابع کانتینری
Single-Purpose Filter	فیلتر تک‌منظوره
Bidirectional Communication	ارتباط دوسویه
Error Condition	وضعیت خطا
Fatal Error	خطای مهلک
Tag	برچسب
Annotation (Java)	حاشیه‌نویسی
Custom Attribute (C#)	صفت سفارشی
Metadata	فراداده
Entry Point	نقطهٔ ورود

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Unstructured Monolith	تک‌پارچهٔ بی‌ساختار
Contract	قرارداد
Deterministic	قطعی
Back-and-Forth Communication	ارتباط رفت‌وبرگشتی
Nondeterministic Workflow	گردش‌کار غیرقطعی
Electronic Data Interchange (EDI)	تبادل الکترونیکی داده
Extract, Transform, Load (ETL)	استخراج، تبدیل و بارگذاری
Telemetry	تله‌متری
Uptime	زمان فعالیت
Topic (Kafka)	موضوع
Event-Driven Architecture	معماری رویدادمحور
Plug-in	افزونه
Microkernel Architectural Style	سبک معماری میکروکرنل